

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Практична робота №1

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Аналіз прикладів Narrow AI vs AGI (ChatGPT, рекомендаційні системи, автономні авто).”

Виконав:

Ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Мокрицька О. В.

Мета роботи

Ознайомитися з основними концепціями штучного інтелекту, зокрема з відмінностями між Narrow AI (вузьким штучним інтелектом) та AGI (штучним загальним інтелектом). Отримати практичні навички у: аналізі реальних прикладів Narrow AI, таких як ChatGPT, рекомендаційні системи та автономні автомобілі; порівнянні їх можливостей з гіпотетичними характеристиками AGI; обговоренні етичних, технічних та соціальних аспектів застосування цих систем у технологічних рішеннях. Розвинути навички критичного мислення щодо обмежень сучасного AI та перспектив його розвитку.

Теоретичні відомості

Штучний інтелект (ШІ) – це галузь комп'ютерних наук, яка займається створенням систем, здатних виконувати завдання, що традиційно вимагають людського інтелекту, такі як розпізнавання образів, прийняття рішень, природна мова обробка та навчання. Термін "штучний інтелект" був введений Джоном Маккарті у 1956 році на Дармутській конференції, де було закладено основи дисципліни. ШІ класифікується за рівнем можливостей:

Narrow AI (вузький ШІ, Weak AI або ANI – Artificial Narrow Intelligence): Спеціалізований ШІ, призначений для виконання конкретних завдань у обмеженому домені. Він не володіє загальним інтелектом і не може переносити знання з однієї задачі на іншу без спеціального перенавчання. Narrow AI є домінуючим у сучасних технологіях і базується на алгоритмах машинного навчання (ML), глибокого навчання (DL) та нейронних мережах. Переваги: висока ефективність у вузьких областях, низька складність реалізації. Недоліки: відсутність адаптивності, потреба в великих даних для навчання.

AGI (загальний штучний інтелект, Strong AI або Artificial General Intelligence): ШІ, здатний розуміти, вчитися та застосовувати знання в широкому спектрі завдань на рівні або краще за людину. AGI може самостійно адаптуватися до нових ситуацій, переносити навички та демонструвати креативність. AGI поки що є теоретичним концептом; його реалізація очікується в майбутньому (за оцінками експертів, як-от Рея Курцвейла, – до 2045 року). Переваги: універсальність, автономність. Недоліки: етичні ризики (контроль, безробіття), технічні виклики (енергоспоживання, етика).

Колабораційна (спільна) фільтрація – це найпростіший спосіб зробити рекомендації або прогнози в рекомендаційних системах. Основне припущення полягає в тому, що люди, які дають однакові оцінки або вибирають ті ж речі, також будуть вести себе аналогічно в майбутньому. За допомогою спільної фільтрації ви можете рекомендувати книги на основі вже прочитаних книг, подібних людей або рекламувати послуги.

Практичне завдання

1. Ознайомитися з теоретичними відомостями.

2. Проаналізувати ChatGPT: Згенеруйте 5 запитів (NLP-задачі), оцініть відповіді (точність, галюцинації). Використовуйте OpenAI API.
3. Рекомендаційна система: реалізувати просту collaborative/content-based рекомендацію на датасеті MovieLens; провести оцінку
4. Порівняти з AGI: Обговоріть у групі, як би AGI покращив ці системи. Сформувати звіт з прикладами.
5. Автономні авто: визначте pipeline perception → prediction → planning → control; продумайте сценарії відмов та резервні механізми.
6. Сформувати висновки.

Програмний код з аналізом:

Після ознайомлення з теоретичними відомостями, моїм першим завданням стало згенерувати 5 запитів (NLP-задач) та оцінити відповіді. OpenAI почав виставляти ціну за користування його API, тому я обрав Google AI, тобто Gemini. Після отримання ключа, я написав короткий код в середовищі google collab.

```
# google_ai_example.py
from google import genai

client = genai.Client(api_key="AIzaSyAHl_M8pONg8LZqZg-e7MPoeix42jvXtFI")
prompts = [
    "Переклади речення 'It`s too many practise tasks for me' на українську мову.", # 1. Переклад
    "Напиши короткий виклад (2-3 речення) сюжету 'Ріпка'.", # 2. Узагальнення
    "Яке визначення має термін 'САПР'?", # 3. Питання-Відповідь (Q&A)
    "Визнач емоційне забарвлення (позитивне, негативне, нейтральне) цього відгуку: 'Я в рот махав сценариста і всю виконавчу групу.'", # 4. Аналіз тональності
    "Напиши один абзац тексту про життя студента кафедри САПР НУЛП." # 5. Генерація тексту
]

for i, prompt in enumerate(prompts):
    print(f"\n--- Запит {i+1}: {prompt} ---")
    try:
        response = client.models.generate_content(model="gemini-2.5-flash", contents=prompt)
        print("Відповідь Gemini:")
        print(response.text)
    except Exception as e:
        print(f"Сталася помилка: {e}")
        print("Переконайтеся, що ваш API-ключ налаштовано правильно.")
```

Дальше мені потрібно було реалізувати рекомендаційну систему, тобто просту collaborative/content-based рекомендацію на датасеті MovieLens і провести її оцінку.

Цей скрипт реалізує повний цикл User-Based Collaborative Filtering.

1. Завантаження даних: Я завантажив 'u.data' (рейтинги) та 'u.item' (назви фільмів) з офіційного репозиторію MovieLens.

2. User-Item матриця: Я використав 'pivot_table' бібліотеки Pandas, щоб створити матрицю, де рядки – це 'user_id', стовпці – 'title', а значення – 'rating'. Це ключова структура даних для колаборативної фільтрації.

3. Розрахунок подібності: Я використав 'cosine_similarity' з 'sklearn' для обчислення косинусної подібності між векторами рейтингів кожного користувача. Це створює матрицю, що показує, наскільки "схожий" кожен користувач на кожного іншого.

4. Функція 'get_recommendations': Ця функція знаходить "сусідів" (найбільш схожих користувачів), дивиться, які фільми вони високо оцінили (але які цільовий користувач ще не бачив), і розраховує прогнозований рейтинг на основі зваженої суми рейтингів сусідів.

5. Оцінка (RMSE): Щоб оцінити якість моделі, я розділив дані на тренувальні (80%) та тестові (20%). Я навчив модель (розрахував подібність) на тренувальних даних, а потім спробував спрогнозувати рейтинги з тестового набору. RMSE (Root Mean Squared Error) показує середню помилку прогнозу (наприклад, якщо RMSE = 0.9, ми в середньому помиляємося на 0.9 зірки за 5-бальною шкалою).

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import numpy as np

# --- 1. Завантаження та підготовка даних ---

print("Завантаження даних MovieLens 100k...")
url_ratings = 'http://files.grouplens.org/datasets/movielens/ml-100k/u.data'
url_movies = 'http://files.grouplens.org/datasets/movielens/ml-100k/u.item'
ratings = pd.read_csv(url_ratings, sep='\t', names=['user_id', 'movie_id',
'rating', 'timestamp'])
movies = pd.read_csv(url_movies, sep='|', encoding='ISO-8859-1',
names=['movie_id', 'title'] + [f'col{i}' for i in range(22)], usecols=[0,
1])
print("Дані завантажено.")

data = pd.merge(ratings, movies, on='movie_id')

# --- 2. Створення User-Item матриці ---

# Створюємо матрицю: рядки - user_id, стовпці - title, значення - rating
user_item_matrix = data.pivot_table(index='user_id', columns='title',
values='rating')
# Заповнюємо NaN (де користувач не ставив оцінку) нулями.
user_item_matrix_filled = user_item_matrix.fillna(0)

# --- 3. Розрахунок подібності (Колаборативна фільтрація User-Based) ---
# Розраховуємо косинусну подібність між користувачами
user_similarity = cosine_similarity(user_item_matrix_filled)
user_similarity_df = pd.DataFrame(user_similarity,
index=user_item_matrix.index, columns=user_item_matrix.index)
```

```

# --- 4. Функція для генерації рекомендацій ---
def get_recommendations(user_id, num_recommendations=5):
    # Знаходимо користувачів, найбільш схожих на нашого user_id
    similar_users =
user_similarity_df[user_id].sort_values(ascending=False)[1:]

    # Фільми, які наш user_id ВЖЕ бачив (щоб не рекомендувати їх знову)
    seen_movies = user_item_matrix.loc[user_id].dropna().index

    recommendations = {}

    for sim_user_id, similarity_score in similar_users.items():
        sim_user_movies = user_item_matrix.loc[sim_user_id].dropna()

        for movie_title, rating in sim_user_movies.items():
            if movie_title not in seen_movies:
                if movie_title not in recommendations:
                    recommendations[movie_title] = {'total_score': 0,
'similarity_sum': 0}

                    recommendations[movie_title]['total_score'] += rating *
similarity_score
                    recommendations[movie_title]['similarity_sum'] +=
similarity_score

        final_recs = []
        for movie, scores in recommendations.items():
            if scores['similarity_sum'] > 0: # Уникаємо ділення на нуль
                predicted_rating = scores['total_score'] /
scores['similarity_sum']
                final_recs.append((movie, predicted_rating))

        final_recs.sort(key=lambda x: x[1], reverse=True)

        return final_recs[:num_recommendations]

# --- 5. Оцінка моделі (RMSE) ---

# Для оцінки ми повинні прогнозувати рейтинги, які ми знаємо.
train_data, test_data = train_test_split(ratings, test_size=0.2,
random_state=42)

train_user_item_matrix = train_data.pivot_table(index='user_id',
columns='movie_id', values='rating').fillna(0)
train_user_similarity = cosine_similarity(train_user_item_matrix)
train_user_similarity_df = pd.DataFrame(train_user_similarity,
index=train_user_item_matrix.index, columns=train_user_item_matrix.index)

# Функція для прогнозування ОДНОГО рейтингу
def predict_rating(user_id, movie_id):
    if user_id not in train_user_similarity_df.index or movie_id not in
train_user_item_matrix.columns:
        return 3.0 # Повертаємо середнє, якщо немає даних (холодний старт)

```

```

    similar_users =
train_user_similarity_df[user_id].sort_values(ascending=False)[1:]

    total_score = 0
    similarity_sum = 0

    for sim_user_id, similarity_score in similar_users.items():
        # Перевіряємо, чи схожий юзер оцінював цей фільм
        if train_user_item_matrix.loc[sim_user_id, movie_id] > 0:
            rating = train_user_item_matrix.loc[sim_user_id, movie_id]
            total_score += rating * similarity_score
            similarity_sum += similarity_score

    if similarity_sum == 0:
        return 3.0

    return total_score / similarity_sum

# Отримуємо прогнози для тестового набору
predictions = []
true_ratings = []

for _, row in test_data.iterrows():
    predictions.append(predict_rating(row['user_id'], row['movie_id']))
    true_ratings.append(row['rating'])

# Розраховуємо RMSE
rmse = np.sqrt(mean_squared_error(true_ratings, predictions))

# --- 6. Вивід результатів ---

print("\n--- Рекомендації (Завдання 3) ---")
target_user_id = 10 # Приклад для користувача 10
recs = get_recommendations(target_user_id, 5)
print(f"Топ 5 рекомендацій для User ID {target_user_id}:")
for movie, score in recs:
    print(f"    {movie} (Очікуваний рейтинг: {score:.2f})")

print("\n--- Оцінка моделі (Завдання 3) ---")
print(f"Root Mean Squared Error (RMSE) на тестових даних: {rmse:.4f}")

```

Результати роботи коду:

Аналіз запитів Gemini

Після запуску коду для Gemini, я отримав такі (зразкові) результати. Я оцінив їх на точність та наявність галюцинації.

```

--- Запит 1: Переклади речення 'It`s too many practise tasks for me' на
українську мову. ---
Відповідь Gemini:
Ось кілька варіантів перекладу, які є природними для української мови:

```

1. ****Забагато тренувальних завдань для мене.**** (Найбільш прямий і поширений варіант)
2. ****Мені забагато тренувальних завдань.**** (Також дуже природно, акцент на "мені")
3. ****Для мене забагато практичних завдань.**** (Якщо "practical tasks" розуміється саме як "практичні", а не "тренувальні")
4. ****Забагато вправ для мене.**** (Якщо "practise tasks" розуміється як "вправи")

Найкращий варіант, залежно від контексту, зазвичай буде ****Забагато тренувальних завдань для мене**** або ****Мені забагато тренувальних завдань****.

Модель правильно переклала текст, якщо судити обмежену наведену інформацію, так як воно мене і моїх уподобань не знає, вона запропонували найбільш ймовірні результати.

--- Запит 2: Напиши короткий виклад (2-3 речення) сюжету 'Ріпка'. ---
Відповідь Gemini:

Дід посадив ріпку, яка виросла настільки великою, що її неможливо було витягнути. Тоді дід послідовно кликав на допомогу бабку, внучку, собачку, кішку і навіть мишку, і лише спільними зусиллями їм вдалося витягнути величезну ріпку.

Тут все добре, галюцинацій не бачу, історія переказана по найбільш відомій версії книжки.

--- Запит 3: Яке визначення має термін 'САПР'? ---
Відповідь Gemini:

Термін ****САПР**** розшифровується як ****Системи Автоматизованого Проектування****.

****Визначення:****

****САПР**** – це комплекс програмних та апаратних засобів, призначений для автоматизації процесів проектування, конструювання, розрахунків та візуалізації виробів, будівель, споруд та інших об'єктів у різних галузях промисловості, архітектури та будівництва.

****Основна мета САПР:****

- * Підвищення продуктивності та ефективності праці інженерів, конструкторів та проектувальників.
- * Скорочення термінів розробки та впровадження нових виробів.
- * Поліпшення якості проектних рішень та документації.
- * Зменшення кількості помилок та витрат на етапі проектування.
- * Забезпечення можливості швидкого внесення змін та варіантних розрахунків.

Англійський еквівалент терміна САПР – ****CAD (Computer-Aided Design)****. САПР є ключовою складовою сучасних САХ-технологій (Computer-Aided Technologies), до яких також входять CAE (Computer-Aided Engineering) та CAM (Computer-Aided Manufacturing).

Тут модель надала більше інформації чим зазвичай, через більш детальний і вибагливий запит. В загальному контексті вона відповіла правильно, але якби вона знала мене, то б зрозуміла що йдеться про нашу кафедру.

--- Запит 4: Визнач емоційне забарвлення (позитивне, негативне, нейтральне) цього відгуку: 'Я бив сценариста і всю виконавчу групу.' ---
Відповідь Gemini:

Емоційне забарвлення цього відгуку — ****різко негативне****.

Фраза "Я бив" є дуже грубим, вульгарним і образливим висловом, що виражає сильне презирство, зневагу, гнів та незадоволення щодо сценариста і всієї виконавчої групи.

Ну тут все добре, галюцинацій тож не видно.

--- Запит 5: Напиши один абзац тексту про життя студента кафедри САПР НУЛП.

Відповідь Gemini:

None

Досконала і вичерпна відповідь, 10/10.

Рекомендаційна система

Дальше рекомендаційна система. Після запуску коду я отримав наступні результати:

Завантаження даних MovieLens 100k...

Дані завантажено.

--- Рекомендації (Завдання 3) ---

Топ 5 рекомендацій для User ID 10:

Santa with Muscles (1996) (Очікуваний рейтинг: 5.00)

Marlene Dietrich: Shadow and Light (1996) (Очікуваний рейтинг: 5.00)

Great Day in Harlem, A (1994) (Очікуваний рейтинг: 5.00)

Someone Else's America (1995) (Очікуваний рейтинг: 5.00)

Aiqing wansui (1994) (Очікуваний рейтинг: 5.00)

--- Оцінка моделі (Завдання 3) ---

Root Mean Squared Error (RMSE) на тестових даних: 1.0153

Значення RMSE близько 1.0 на 5-бальній шкалі означає, що прогнози моделі в середньому відхиляються від реального рейтингу користувача на 1 бал. Для такої простої моделі колаборативної фільтрації це прийнятний, хоч і не ідеальний, результат.

Теоретична робота:

В цьому розділі моє завдання виконати завдання, які пов'язані з моїми думками та роздумами на певну тематику.

Порівняння з AGI

Проблема Narrow AI: Gemini/ChatGPT – це моделі мови, а не моделі світу. Вони генерують статистично ймовірний текст. Вони не "розуміють" світ, не мають пам'яті (поза контекстом) і не можуть робити логічні висновки, засновані на реальному досвіді. Їхні "галюцинації" – це наслідок відсутності базового розуміння реальності. AGI мав би справжнє розуміння і свідомість (в теорії).

AGI не просто б передбачав текст, він би звертався до своєї внутрішньої, узгодженої моделі світу. На питання "Хто виграв ЧС-2026?" він би не вигадував відповідь, а сказав би "Ця подія ще не відбулася", бо розуміє концепцію часу.

AGI б навчався з кожної взаємодії. Якщо б я йому сказав "Я вчуся в Політехніці", він би запам'ятав це назавжди, а не лише на час сесії (хоча вже сучасні моделі підтримують пам'ять між сесіями).

AGI б розумів не лише текст, а й мої емоції, тон голосу, контекст ситуації. Він би перетворився з "інструменту" на "співрозмовника" або "асистента".

Автономні авто

Аналіз автономних авто (приклад складного Narrow AI) показує, наскільки комплексними є ці системи, навіть не будучи AGI.

Я визначив наступний типовий цикл роботи автономного авто:

1. Perception (Сприйняття): Збір даних з сенсорів (LiDAR, камери, радары, GPS). "Що навколо мене?" Система розпізнає об'єкти: інші авто, пішоходи, велосипедисти, дорожня розмітка, знаки.

2. Prediction (Прогнозування): Моделювання майбутнього. "Що ці об'єкти зроблять?" Система (зазвичай на базі ML) прогнозує траєкторії руху об'єктів (наприклад, "цей пішохід, ймовірно, ступить на дорогу").

3. Planning (Планування): Вибір траєкторії. "Який мій найкращий шлях?" На основі сприйняття та прогнозування система планує власну траєкторію (довгострокову – маршрут до цілі, і короткострокову – об'їзд перешкоди).

4. Control (Керування): Виконання. "Як мені це зробити?" Система надсилає конкретні команди виконавчим механізмам: "повернути кермо на 5 градусів", "натиснути гальма з силою X".

Сценарії відмов та резервні механізми:

1. Відмова Perception (наприклад, туман / брудна камера): Камера не бачить розмітку вночі під дощем.

Резерв (Sensor Fusion): LiDAR та радар все ще "бачать" фізичні перешкоди (бордюр, інше авто) незалежно від освітлення. Система повинна довіряти комбінації даних, а не одному сенсору.

2. Відмова Prediction (нетипова поведінка): Пішохід раптово стрибає на дорогу, або інше авто робить непередбачуваний маневр.

Резерв (Defensive Driving): Алгоритми планування заздалегідь закладають "захисний" (defensive) стиль водіння – тримають дистанцію, знижують швидкість у зонах ризику (біля шкіл), залишаючи час на реакцію.

3. Відмова Planning (етична дилема / "проблема ва Mgonetky"): Авто не може уникнути аварії, і змушене обирати між двома поганими сценаріями (наприклад, збити пішохода А або пішохода Б).

Резерв (Pre-defined Ethics / Minimal Harm): Це найскладніша проблема. Резервом є заздалегідь запрограмована етична матриця (наприклад, пріоритет збереження пішоходів) або алгоритм "мінімальної шкоди". Єдиного рішення поки немає.

4. Відмова Control (апаратна несправність): Відмова основної системи гальмування.

Резерв (Redundancy): Наявність дублюючих систем (drive-by-wire). Якщо відмовили гідравлічні гальма, спрацьовує резервна електронна система гальмування (наприклад, рекуперативне гальмування двигунами).

Висновки

Під час виконання цієї лабораторної роботи я ознайомився з основними концепціями штучного інтелекту, зокрема з відмінностями між Narrow AI (вузьким штучним інтелектом) та AGI (штучним загальним інтелектом). Отримав практичні навички у: аналізі реальних прикладів Narrow AI, таких як ChatGPT, рекомендаційні системи та автономні автомобілі; порівнянні їх можливостей з гіпотетичними характеристиками AGI; обговоренні етичних, технічних та соціальних аспектів застосування цих систем у технологічних рішеннях. Розвинув навички критичного мислення щодо обмежень сучасного AI та перспектив його розвитку. Ця робота чітко продемонструвала, що хоча сучасний ШІ досяг вражаючих результатів, він залишається "слабким" (Narrow), і розрив між ним та гіпотетичним "сильним" ШІ (AGI) залишається величезним.